5-1 · 분산 처리와 Spark 아키텍처 · 정리·회고 0.5H

정리·회고

개념 아홉 개를 한 표로, 5-2 클러스터 준비 확인

이 시간의 목표

30분 뒤에는

① 개념 아홉 개를 「한 줄 정의 · UI 어디」로 정리한다

② 5-2 를 위해 Spark 이미지와 Docker 메모리를 확인한다

③ 체크리스트 열한 개로 오늘 한 것을 스스로 점검한다

④ 자주 겪는 오류 다섯 가지의 원인과 해결을 안다

파티션 수와 cache 시간은 환경마다 다르게 나옵니다 — 내 숫자를 메모해 두세요

오늘의 요약

개념 아홉 개 — 한 줄 정의와 UI 위치

개념	한 줄 정의	UI 에서 어디
드라이버	계획을 세우고 결과를 모은다	Executors 탭의 driver
익스큐터	실제 계산을 한다	Executors 탭(5-2 에서 늘어남)
transformation	계획만 쌓는다	Job 이 안 생김
action	실행을 시작한다	Job 이 생김(하나 이상)
Job	action 이 만든다(여럿도)	Jobs 탭
Stage	셔플을 경계로 나뉜다	Job 상세 · Stages 탭
Task	파티션 하나당 하나	Stage 상세
셔플	노드 간 데이터 재배치	계획의 Exchange
DAG	실행 계획 그림	Job 상세 DAG Visualization

action 하나가 Job 여럿을 만들기도 한다 — Stage 는 셔플 경계, Task 는 파티션 하나당 하나

다음 차시(5-2) 준비

이미지와 Docker 메모리를 확인한다

1 셸은 이미 나왔다

실습 7단계에서 exit() — 4040 이 닫혀 있다

2 이미지 확인 — 5-2 에서 그대로 쓴다

```
docker images apache/spark
```

3 메모리 확인 — 5-2 는 컨테이너가 3개 이상

```
docker info ¦ grep -i "total memory"
```

확인 포인트

apache/spark:3.5.3 · 1.56GB

Total Memory 4GiB 이상

> 실측 7.749GiB — 8GB 안팎 권장 · 모자라면 Docker Desktop → Settings → Resources(WSL2 는 .wslconfig 의 memory)

오늘의 체크리스트 1~6

열한 개를 스스로 확인해 보세요

1. 스케일업과 스케일아웃의 차이를 설명할 수 있다
2. 「한 대로 되면 한 대로 한다」는 판단 기준을 말할 수 있다
3. PySpark 셸을 컨테이너로 기동했다
4. 드라이버/익스큐터의 역할을 비유 없이도 설명할 수 있다
5. transformation 단계에서 Job 이 안 생김을 UI 로 확인했다
6. action 호출 후 Job → Stage → Task 계층을 UI 에서 열어 봤다

UI 는 셸을 띄운 순간부터 열려 있다 — action 전에도 Jobs 탭이 보인다

오늘의 체크리스트 7~11

열한 개를 스스로 확인해 보세요

7	explain() 을 아래에서 위로 읽고 앞당겨진 조건과 Exchange 를 찾았다
8	다른 DataFrame 의 컬럼을 쓰면 action 전에 AnalysisException 을 봤다
9	groupBy 의 「나누기」·「모아서 세기」를 Job·Stage(skipped)로 확인했다
10	파티션 수 = 첫 Stage Task 수, 셔플 뒤 Task 는 AQE 가 줄임을 봤다
11	cache 전후 시간을 재 보고 Storage 탭에서 캐시를 확인했다

자주 겪는 오류

증상으로 찾는 원인과 해결

증상	원인	해결
! 컨테이너 종료 · OutOfMemoryError	Docker 메모리 부족 · 큰 데이터에 collect()	✓ 메모리 4GB 이상 · show()·limit() 으로
! localhost:4040 이 안 열린다	-p 4040:4040 누락 · 셸을 이미 나옴	✓ 포트 매핑을 넣어 다시 기동
! Bind for 0.0.0.0:4040 failed	다른 창의 pyspark 셸이 4040 을 쓰는 중	✓ 앞 셸 exit() · 또는 -p 4041:4040
! withColumn·filter 가 아무 출력 없음	오류가 아니라 지연 평가	✓ show()·count() 같은 action 을 붙인다
! 셸을 나갔더니 4040 이 안 열림	UI 는 앱이 살아 있는 동안만 뜬다	✓ 셸 재기동 = 새 UI · History Server(심화)

> action 을 안 했어도 셸이 떠 있으면 Jobs 탭은 열린다

예상 질문

자주 나오는 질문 네 가지

Q Pandas 를 쓰면 안 되나요?	**A** 메모리에 들어가면 Pandas 가 훨씬 빠르고 간단 — 작은 데이터에 Spark 는 오히려 느리다(분산 비용)
Q local[*] 인데도 분산인가요?	**A** 코어 수만큼 나눠 병렬로 — 여러 대는 아니지만 구조가 같다, 5-2 에서 진짜 클러스터
Q cache 를 항상 쓰면 좋은 것 아닌가요?	**A** 메모리를 차지한다 — 한 번만 쓰면 손해, 「두 번 이상 쓰는가」로 판단
Q Spark UI 는 실무에서 얼마나 쓰나요?	**A** 아주 많이 — 느린 잡의 원인을 찾는 거의 유일한 도구, 오늘의 세 계층이 그 기초

심화 과제 (선택)

더 해 볼 것 네 가지

1. **같은 count() 두 번 + cache()**
   UI 에서 두 Job 의 실행 시간을 비교하고, cache() 뒤 두 번째가 어떻게 달라지는지
2. **기본 파티션 수의 가설**
   df.rdd.getNumPartitions() 값이 어떻게 정해졌을지 — 답은 5-5 에서
3. **AQE 끄고 groupBy 를 새로 만들기**
   Job 다섯에 Task 1·4·20·100·75(합 200) · shuffle.partitions 8 이면 1·4·3
4. **join 하나의 explain()**
   Exchange 가 몇 개 생기는지 세어 본다

> **AQE 끄기: spark.sql.adaptive.enabled false — 같은 DataFrame 변수를 다시 show() 하면 앞 계획을 써서 안 바뀐다**

다음 회차 예고 · 10-26(월)

5-2 · Docker Compose 로 Spark 마스터-워커 구축

01

이론 1H

Spark 클러스터
배포 구조 이해

02

실습 1.5H

Compose 로 마스터-
워커 클러스터 구축

03

실습 1H

spark-submit 으로
클러스터에 잡 제출

04

정리 0.5H

정리·회고
이론·실습

사전 준비: Docker Desktop 실행 + docker compose version 정상 출력 확인 · 5-1 의 단일 컨테이너 실습은 종료

5-1 · 정리·회고 0.5H 마무리

다음 회차는 5-2 Spark 마스터-워커 클러스터

compose 파일 하나로 마스터 1 + 워커를 띄우고, spark-submit 으로 잡을 제출해 마스터 UI 에서 추적합니다

- ✓ 마스터와 워커의 역할 구분
- ✓ Compose 로 클러스터 기동 · 워커 등록 확인
- ✓ spark-submit 으로 애플리케이션 제출
- ✓ 마스터 UI(8080)에서 워커 상태 · 실행 이력

준비할 것

compose version 확인 · 셸 종료